



基于多目标跟踪 (MOT) 的自动驾驶激光 SLAM 系统

ME5400 项目组

2026 年 4 月 14 日

目录

1 引言与背景	1
2 研究现状分析	3
2.1 从动态点剔除到动态感知估计	4
2.2 近期 SLAMMOT 系统与向联合优化的推进	5
2.3 纯 3D MOT 背景与跟踪主干的选择	6
2.4 对比分析与存在的差距	6
3 整体系统架构、PointPillars / MCTrack 与数据流	6
3.1 FAST-LIO 前端	7
3.2 PointPillars 检测模块	8
3.3 MCTrack 模块与数据流细节	10
3.4 时间对齐、坐标系与失效模式	11
4 工程实现与系统集成	11
4.1 整体代码组织	11
4.2 在线系统设计	12
4.3 结果归档与可复现的实验管线	13
4.4 ROS 话题、消息接口与集成困难	14
5 动态权重优化	14
5.1 实现与交互逻辑	14
5.2 权重映射策略	15
5.3 软降权与硬剔除的权衡	15
6 联合后端优化	15
6.1 优化目标与残差建模	15
6.2 失效模式分析：以 Sequence 0009 为例	16
7 实验分析	16
7.1 定量性能总结	17
7.2 分类场景讨论与失效感知分析	17
7.2.1 高动态改善场景：Sequence 0003 与 0013	17

目录	2
7.2.2 中等动态稳定场景: Sequence 0006, 0007, 0010	18
7.2.3 失效案例与漂移挑战: Sequence 0009 与 0020	18
7.3 因果分析: 增益究竟来自何处?	18
8 结论与未来计划	18
附录: 在线实验全量图片	20

摘要

本报告详尽阐述了 ME5400 课程项目——面向动态场景下的激光 SLAM 与多目标跟踪 (MOT) 系统。该系统集成了用于激光惯性自运动估计的 FAST-LIO、用于 3D 目标检测的 PointPillars 以及用于在线轨迹关联和运动状态估计的 MCTrack。其核心思想是构建一个双向闭环而非单向串联：FAST-LIO 提供的位姿稳定了 MCTrack 的关联，而跟踪到的目标状态被转化为 FAST-LIO 中的软动态权重，并作为对象一致性因子引入仅包含自车位姿 (ego-only) 的滑窗后端中。

所有定量讨论均基于 Results/ 目录下 7 个 KITTI 跟踪序列 (0003, 0006, 0007, 0009, 0010, 0013, 0020) 的归档在线实验结果。报告强调算法设计意图与工程实现细节，包括 ROS 接口、时间同步、位姿缓冲、标准结果归档以及失效模式分析。在所有 7 个序列中，前端配置 FAST-LIO+MCTrack 在 ATE RMSE 指标上均始终优于纯 FAST-LIO 基线，表明最可靠的增益源自“位姿辅助跟踪”与“跟踪反馈式动态降权”形成的前端闭环。联合后端虽在部分漂移严重的序列上有所帮助，但其稳定性较弱，在速度外推、时间戳对齐或目标门控不可靠时可能失效，序列 0009 便是最典型的例子。本报告的总体结论采用了审慎的态度：本项目并非追求 SOTA 的联合 SLAMMOT 系统，而是一个可复现的工程原型，它验证了定位-跟踪反馈闭环的可行性，并为未来向以对象为中心 (object-centric) 的联合优化转型奠定了基础。

关键词： FAST-LIO; PointPillars; MCTrack; 动态场景 SLAM; 多目标跟踪; 面向对象的感知; ROS 重现性

1 引言与背景

在动态道路场景中进行定位与感知，往往面临着一对此消彼长且难以调和的矛盾。激光 SLAM 及激光惯性里程计的基础假设是所观测到的大部分几何特征是静态、可重复见且可长期复用的。其前端扫描匹配、局部地图维护及后端一致性保障，均高度依赖于“当前时刻观测到的点所对应的表面在后续帧中依然具有几何意义”这一前提。然而，自动驾驶感知系统最关注的对象（如周边车辆、行人、骑行者等）却恰恰违背了这一假设。换言之，在语义层面对决策至关重要的区域，在几何层面往往对传统的 SLAM 系统构成了巨大威胁。这一冲突催生了大量关于动态场景 SLAM 的研究，从早期的通过检测并过滤运动区域的视觉系统，到近期能够联合推理自运动与目标轨迹的 SLAMMOT 管线 [3, 4, 5, 7, 10]。

动态目标对激光 SLAM 的负面影响并非仅仅是“运动点是离群值”那么简单，而是通过至少三个耦合的退化通道起作用。首先是**地图污染 (map contamination)**：运动车辆的点被当作静态表面插入局部或全局地图中，导致系统积累“鬼影”结构、双重墙体或拖尾，误导后续帧的配准。其次是**配准退化 (registration degradation)**：即使这些点尚未永久融合进地图，如果当前帧配准残差中有很大大比例由运动物体产生，局部优化器就会被迫去拟合不一致的几何结构，从而使当前帧的位姿估计产生偏差。最后

是**漂移累积 (drift accumulation)**：随着时间推移，上述两个问题会互相放大——微小的帧间修正偏差会扭曲局部地图，进而导致后续修正效果进一步恶化，最终产生持久的轨迹偏移。因此，动态场景不仅仅是增加了随机噪声，而是改变了优化问题本身的结构。

现有的动态场景 SLAM 系统通常采用以下三种策略之一。第一类尝试通过语义分割、运动一致性测试或鲁棒损失函数完全剔除动态点。当动态目标占视野比例较小时，该策略往往非常有效，但在大型车辆占主导的拥挤交通场景中，这可能会丢弃过多有用的几何结构。第二类尝试更显式地分离静态层与动态层，例如在稠密 RGB-D 管线中采用加权背景融合，或在视觉 SLAM 中进行语义前景过滤 [3, 4, 5]。第三类也是更新的一类，将目标状态视为一等变量，追求自运动与被跟踪目标之间的某种程度的联合优化 [6, 7, 8, 9, 10]。对于本项目而言，实际的问题在于一个学期规模的工程系统应该如何切入这一设计空间。

多目标跟踪 (MOT) 面临着与之相反但同样重要的依赖关系。跟踪器通常被描述为对“检测”进行操作，但仅靠 3D 检测不足以跨帧提供稳定的运动估计。当自车本身在移动时，每一个检测结果都必须经过自车运动补偿 (ego-motion compensation) 才能被正确解读；否则，一个在世界系下静止的目标会看起来在移动，而一个平稳移动的目标则可能出现跳变。这不仅影响关联，还影响融合速度、加速度及跟踪置信度等衍生指标。如果位姿流存在噪声，跟踪器在世界系下的运动估计也会随之不准。反之，如果跟踪器无法维持连贯的 ID 和速度轨迹，它也无法为定位堆栈提供可靠的反馈。因此，两者之间的关系本质上是双向的：更好的定位能改善跟踪，而更好的跟踪可以通过识别哪些点应被赋予较低权重，或通过提供额外的目标级约束来改善定位。

本项目正是植根于这一双向直觉。系统采用 FAST-LIO [1] 作为激光惯性定位主干，PointPillars [2] 作为 3D 检测器，MCTrack 作为在线跟踪器以生成目标 ID、融合速度、加速度及轨迹长度。FAST-LIO 发布优化里程计与 IMU 预测运动。桥接节点将这些输出转换为 MCTrack 所需的位姿格式。PointPillars 按时间戳发布 3D 检测消息。MCTrack 消费检测与位姿，执行经运动补偿的关联，并发布跟踪目标。这些跟踪目标随后通过两种方式被利用：一是转化为软动态权重，在下一帧 FAST-LIO 扫描中降低动态包围盒内点的权重；二是送入仅含自车位姿 (ego-only) 的联合后端，测试稳定的目标观测能否为自车轨迹提供额外的滑窗约束。这一设计虽然尚未构成统一的因子图，但已形成了一个完整的信息闭环。

在工程实践中，我们有两项设计选择需要简要说明。首先是采用**软降权 (soft down-weighting)**而非硬剔除，因为硬删除在误报、框抖动、遮挡或暂时静止的情况下显得过于脆弱。其次是选择从一个**松耦合的工程闭环**起步，而非紧耦合的对象中心图。完整的联合图在理论上更优雅，但其所需的因子设计、置信度建模、边缘化处理及调试工作量远超本课程项目的周期。

据此，本项目追求四个关联的目标。**研究目标**是测试定位与跟踪在不构建完整统一优化器的情况下，能否在动态驾驶场景中形成有意义的反馈闭环。**工程目标**是构建一个

可运行的 ROS 管线，其组件、消息契约、启动顺序、结果日志及评估脚本均是显式且可复现的。**评估目标**是确定闭环中哪一部分提供了最稳定的收益：是前端权重环节、后端修正环节，还是两者兼有。**诊断目标**是识别阻碍当前系统表现得像一个鲁棒的联合 SLAMMOT 方案的具体失效模式。

具体而言，本报告及实现的贡献可总结如下：

- 实现了 FAST-LIO、PointPillars、MCTrack 与 ego-only 后端的完整 ROS 集成，采用项目定义的标准消息接口而非临时私有接口。
- 构建了前端闭环：FAST-LIO 位姿稳定了 MCTrack，而 MCTrack 的输出在 FAST-LIO 预处理中转化为点级软权重。
- 建立了可复现的实验框架，支持基线与优化运行的一键切换、Results/ 下的标准归档、TUM 格式轨迹记录及统一的定量评估。
- 通过失效感知分析表明，前端闭环是最稳定的改进源，而当前的后端在速度外推及时间不确定性面前仍显脆弱。

选择 KITTI Tracking 作为主要验证数据集是深思熟虑的结果。KITTI 为真实的城市驾驶场景提供了同步的激光雷达、IMU、标定信息及自车运动真值 [?]。其 Tracking 切片包含测试定位-跟踪闭环所需的动态车辆、轨迹变化及背景演变。本文使用的 7 个在线序列 (0003, 0006, 0007, 0009, 0010, 0013, 0020) 涵盖了不同的漂移水平、动态强度及相对轨迹跨度。本报告并非追求全数据集的榜单排名，而是分析该系统在那些日志、图表及轨迹文件均可直接复现和检查的序列上的表现。

这一立场引出了一个重要的定位说明：**本报告并非旨在声称达到了 SOTA 性能。**它记录的是一个验证了双向定位-跟踪闭环可行性、展示了完整实验管线并使失效案例透明化的工程原型。对于课程项目而言，这种集成质量、可重复性及诊断能力本身就是极具价值的成果。一个端到端运行、可多序列回放并能准确暴露失效原因的系统，比一个未经充分调试、仅在表面上显得“紧耦合”的方案更能为后续研究打下坚实基础。

2 研究现状分析

为了更准确地定位本项目，必须从两个维度讨论相关工作。第一类是**动态场景 SLAM**，其重点在于检测、过滤或联合建模运动区域，以保持定位与建图的稳定性。第二类是**面向自动驾驶的 3D MOT**，其重点在于在自车运动下维持准确的目标轨迹和运动状态。本项目处于这两者的交汇点：它既不是单纯的动态物体过滤系统，也不是一个完整的以对象为中心的联合优化器。因此，一个有意义的对比需要考虑传感器配置、前端形式、耦合程度、动态物体处理策略、实时性倾向以及工程复杂度。

表 1: 与本项目相关的代表性动态场景 SLAM 及 3D MOT 工作对比

工作	传感器 / 前端	耦合方式	动态处理 / 实时性	与本项目的关联
StaticFusion	RGB-D; 稠密直接法 SLAM	联合估计相机运动与分割	加权背景融合; 实时稠密建图 [3]	展示了采用软降权及保留背景结构而非单纯硬剔除的价值。
DS-SLAM	视觉; ORB-SLAM2 + 语义线程	松耦合多线程设计	语义过滤加运动一致性; 面向实时运行 [4]	证明了模块化、多线程的工程结构仍能提高动态场景下的鲁棒性。
DynaSLAM	视觉; ORB-SLAM2 家族	侧重过滤, 恢复静态图	动态检测加背景修复; 较慢但精确 [5]	阐明了将动态目标主要视为待移除区域 s 的优势与局限。
Conf SLAM-MOT	激光; 激光里程计	紧耦合因子图	优化内部的置信度引导关联 [6]	启发了后续基于置信度的对象因子权重设计。
LIMOT	激光 + 惯性; 自研 LIO	紧耦合滑窗因子图	实时联合激光惯性里程计与 MOT [7]	是本项目在超越话题级耦合后, 最接近的高层目标架构。
IMM-SLAMMOT	双目视觉; ORB-SLAM2	紧耦合 BA	显式的目标运动模型切换 [8]	强调了在交通场景中, 单一的恒定速度模型往往是不够的。
Online Dynamic SLAM	双目视觉; 自研 VO	紧耦合增量平滑	基于 iSAM2 的在线动态优化 [9]	指出了为什么增量式后端对于可扩展性和稳定性至关重要。
DL-SLOT	激光; 激光里程计	紧耦合协同图优化	定位与跟踪之间的滑窗协同 [10]	为激光侧的窗口管理和对象因子集成提供了相关参考。
AB3DMOT	激光检测; 简单 KF 跟踪器	无 SLAM 耦合	3D 卡尔曼滤波加匈牙利匹配; 极速 [?]	代表了许多实际 MOT 管线背后的轻量级跟踪哲学。
CenterPoint	激光; 基于中心的检测/跟踪	检测-跟踪集成, 无 SLAM 耦合	利用中心速度实现简单且强大的跟踪 [?]	在讨论 MCTrack 的检测器-跟踪器设计选择时作为重要的对比基线。

2.1 从动态点剔除到动态感知估计

StaticFusion、DS-SLAM 和 DynaSLAM 共同定义了动态场景 SLAM 的早期演进 [3, 4, 5]。尽管它们使用的是 RGB-D 或视觉前端而非激光惯性估计, 但它们确立了三个至今依然适用的理念: 应显式检测动态区域; 应保守地保留背景结构而非盲目删除; 模块化系统在不将所有内容合并为单一优化器的情况下仍能改善鲁棒性。相对于本项目, 它们的共同局限在于动态物体主要被视为需要抑制或避开的区域, 而非具备运动属性、后续可能帮助定位的跟踪实体。当前系统与之不同的是, 在过滤之后依然保持了对象级信息的存续。

2.2 近期 SLAMMOT 系统与向联合优化的推进

Conf SLAMMOT、LIMOT、IMM-SLAMMOT、Online Dynamic SLAM 和 DL-SLOT 集体展示了该领域当前的发展方向 [6, 7, 8, 9, 10]。这些工作不再将 SLAM 和 MOT 视为孤立的黑盒。相反，它们共享状态、关联置信度、运动先验或优化窗口。最重要的共同趋势是从“先定位、后跟踪”的流水线转向一种对象状态、置信度与自运动在同一估计过程中互相补全的联合系统。

Conf SLAMMOT 特别值得关注，因为它强调了优化过程内部的置信度引导关联 [6]。这对本项目很有意义，因为目前的后端仍使用相对简单的权重（主要基于检测得分 and 轨迹长度）。Conf SLAMMOT 传达的底层信息是：置信度不应仅仅是保留或丢弃对象的阈值，它应该影响对应因子的强度。这正是本项目后端如果要提高在不稳定序列上的鲁棒性所需演进的方向。

LIMOT 可能是本项目预想的长期目标最接近的参考 [7]。它在一个紧耦合的滑动窗口框架内集成了激光惯性里程计与多目标跟踪，是当前基于话题设计的直接对照。与 LIMOT 相比，本项目目前的统一性明显不足：FAST-LIO、PointPillars、MCTrack 和后端仍然通过消息交换而非共享单一状态图。然而，这种对比并非纯粹负面的。由于本项目已经在模块边界发布了必要的信息（包括带时间戳的位姿、雷达系对象观测、世界系跟踪状态、得分与运动属性），它实际上已经组装了紧耦合公式所需的大部分数据。

IMM-SLAMMOT 通过强调动态目标的多种运动模型贡献了不同的经验 [8]。交通参与者并不都遵循单一整齐的匀速模式。汽车会停止、加速和转弯；行人的行为更难预测。本项目继承了 MCTrack 在线模式下的实用匀速行为，这对于第一个工程原型是足够的，但一旦这些跟踪速度被用作后端约束，就会暴露出局限。在某些序列（尤其是 Sequence 0009）中观察到的性能下降，一定程度上可以通过这一视角来理解：后端信任了基于过于简单模型生成的运动外推。

Online Dynamic SLAM 值得注意，因为它专注于动态场景中的增量平滑与建图，而非仅仅是批处理式的滑动窗口 [9]。其对本项目的启发不在于具体的传感器模态（它是视觉而非激光惯性），而在架构设计经验上：一旦动态信息进入估计问题，窗口管理、增量更新和计算可扩展性就会成为设计的核心。目前 ME5400 的后端仍使用通过非线性最小二乘重复求解的 ego-only 小窗口。这是一个合理的中间阶段设计，但尚未触及真正在线联合 SLAMMOT 系统必须解决的长效扩展问题。

DL-SLOT 在激光雷达侧极具启发性，因为它探索了动态激光 SLAM 与 3D 目标跟踪之间的协同图优化 [10]。相对于本项目，DL-SLOT 代表了相同通用目标的更成熟版本：它不在外部接口层对话，而是在优化层共享信息。这使得 DL-SLOT 成为后续后端演进的有用指南，特别是关于对象相关约束应如何在窗口中调度、加权和边缘化。

2.3 纯 3D MOT 背景与跟踪主干的选择

MOT 文献之所以重要，是因为对象信息的可用性取决于这些信息是如何产生的。AB3DMOT 和 CenterPoint 代表了两种实用的设计哲学 [?, ?]：一种强调在检测之上进行轻量级的卡尔曼式跟踪，另一种则将更多运动信息推入射向器-跟踪器内部。对于本项目而言，关键的对比不在于哪种基准测试最强，而在于哪种接口对下游最有用。MCTrack 的吸引力在于它暴露了轨迹长度、融合速度、速度变化、加速度及稳定 ID，所有这些都能直接复用于动态降权及后端门控。

2.4 对比分析与存在的差距

从这些工作中可以归纳出三个宏观观察。首先，该领域正朝着更紧密的耦合方向发展：最强大的近期系统不再保持 SLAM 和 MOT 的完全分离。其次，动态物体处理仍主要集中在过滤、关联 and 对象状态估计；即使是联合系统也并不总是能将动态物体作为自车定位的几何支撑。第三，实时性与工程复杂度依然是决定性因素，因为随着耦合程度增加，同步、置信度建模与稳定优化变得更加困难。

相对于上述研究格局，本项目占据了一个有意为之的中间位置。它已经实现了一个真实的双向反馈闭环，因此超越了单向级联管线；但它仍然是一个侧重工程的松耦合原型，而非完整的以对象为中心的联合优化器。其价值在于解决了许多方案中隐晦提及的接口与复现性问题：带时间戳的消息、同步、坐标变换、实验编排、标准化日志以及多序列失效分析。存在的差距是明确的：系统已经具备了未来联合优化器所需的信息流，但尚未具备充分利用这些流的状态表示与因子设计。

3 整体系统架构、PointPillars / MCTrack 与数据流

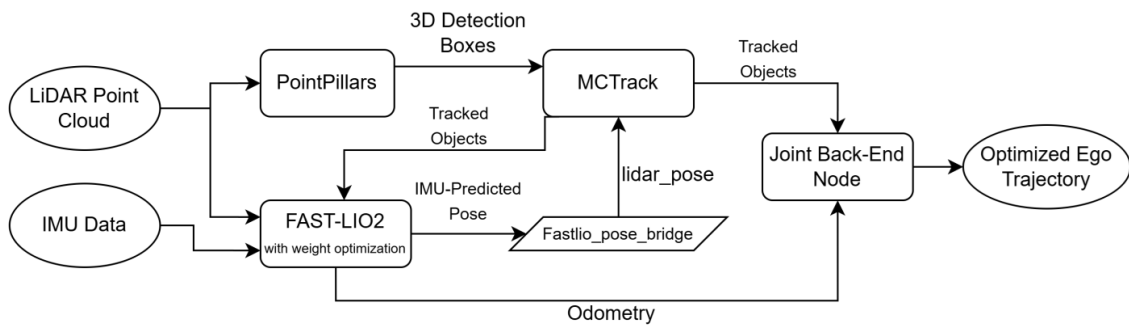


图 1: 整体系统架构。可以将其解读为感知先行、跟踪对齐紧随其后、联合修正最后的三个层次的过程。

图 1 作为一个数据依赖图比作为静态方框图更容易理解。第一层执行感知与自运动估计：FAST-LIO 发布位姿，而 PointPillars 从相同的激光雷达输入发布 3D 检测。第

二层执行时空对齐：fastlio_pose_bridge.py 将 FAST-LIO 里程计转换为 MCTrack 所需的激光位姿接口，MCTrack 将位姿与检测对齐以形成连贯的目标轨迹。第三层执行反馈与修正：跟踪到的目标被送回 FAST-LIO 开启动态点降权，并送往自车后端进行窗口化轨迹修正。

这种组织形式是必要的，因为各模块运行在不同层级上。FAST-LIO 处理原始几何与惯性传播，PointPillars 处理学习到的目标假设，MCTrack 处理时间关联，后端处理短时域的自车修正。保持这些阶段显式化有助于避免阻塞行为，并使增益或失效能够更容易地追溯到具体接口。

如果从数据流视角理解系统，过程可总结为以下七步：

1. 传感器首先获取原始激光点云与 IMU 数据,并同时输入 FAST-LIO 和 PointPillars。
2. PointPillars 产生当前帧的 3D 检测，指出附近存在哪些目标。
3. FAST-LIO 输出当前自位位姿，并持续提供高频运动预测。
4. 位姿桥接节点将这些定位输出转换为跟踪模块直接可用的格式。
5. MCTrack 结合“观测到了什么”与“自车如何运动”来完成目标关联、轨迹更新及速度估计。
6. 跟踪输出反馈给 FAST-LIO，以便对可能属于动态目标的点进行降权。
7. 同样的跟踪输出也被送入联合后端，以进一步精修自车轨迹。

这一七步描述已经说明了为什么系统必须组织为一个有向但延迟的闭环。轨迹在检测关联之前无法存在，动态点权重在跟踪消息存在之前也无法计算。因此，时刻 t 产生的跟踪状态影响的是后续扫描而非当前扫描。这一帧的延迟并非缺陷，而是保证闭环能够在线运行且不产生循环阻塞的关键。

3.1 FAST-LIO 前端

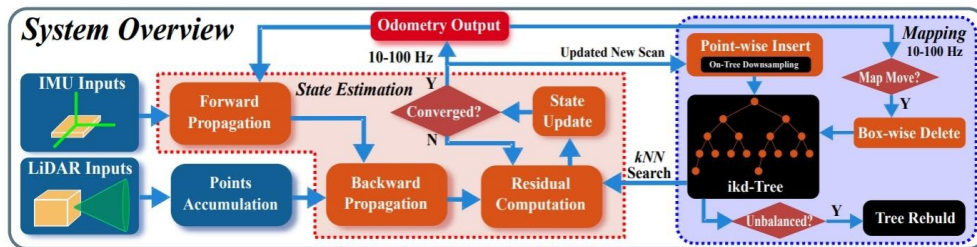


图 2: FAST-LIO2 前端架构。本项目中，来自 MCTrack 的对象信息被注入点云预处理管线，实现对动态点的自适应降权。

FAST-LIO 是整个系统的定位主干，也是上述数据流中的起点。它直接消费来自 KITTI rosbag 的激光点云与 IMU 数据，内部使用紧耦合的激光惯性估计管线输出稳定的自车位姿。本项目中，FAST-LIO 不仅产生全局一致的 /Odometry，还发布 /Odometry_imu_predicted 作为高频预测位姿，为后续跟踪与后端优化提供统一参考。

从模块职责看，FAST-LIO 在系统中扮演三个角色。首先，它为 MCTrack 提供可靠的自车位姿，使得 3D 检测在跨帧关联前能完成自运动补偿。其次，它为联合后端提供原始里程计约束，使后端优化不至于偏离前端估计过远。第三，通过 preprocess.cpp 及 laserMapping.cpp 中的跟踪对象回调路径，它接收 /mctrack/tracked_objects，将其转换为内部 DynamicObject 结构，并将目标运动属性映射为点级权重。因此，FAST-LIO 既是自运动信息的源头，也是对象反馈最直接地改变扫描级几何处理的地方。

从算法角度看，FAST-LIO 前端可以理解为一个结合了 IMU 传播与激光几何修正的迭代滤波器。设系统状态为 $\mathbf{x}_k$ ，IMU 输入为 $\mathbf{u}_k$ 。预测步可写为：

$$\mathbf{x}_{k+1|k} = f(\mathbf{x}_{k|k}, \mathbf{u}_k) + \mathbf{w}_k, \quad \mathbf{P}_{k+1|k} = \mathbf{F}_k \mathbf{P}_{k|k} \mathbf{F}_k^\top + \mathbf{Q}_k, \quad (1)$$

其中 $\mathbf{P}$ 为状态协方差， $\mathbf{F}_k$ 为线性化状态转移矩阵。对于当前帧中的第 j 个点，FAST-LIO 进一步利用局部地图中的参考平面构建点到面的几何残差：

$$r_j = \mathbf{n}_j^\top (\mathbf{R}(\mathbf{x}) \mathbf{p}_j + \mathbf{t}(\mathbf{x}) - \mathbf{q}_j), \quad (2)$$

并通过迭代更新来最小化：

$$\delta \mathbf{x}^* = \arg \min_{\delta \mathbf{x}} \left(\sum_j \|r_j\|_{\mathbf{P}_{k+1|k}}^2 + \|\delta \mathbf{x}\|_{\mathbf{P}_{k+1|k}}^2 \right). \quad (3)$$

这些公式描述了 FAST-LIO 在本项目中的核心作用：IMU 提供高频连续预测，激光点云残差将位姿拉回几何一致的解。这就是为什么 FAST-LIO 既能稳定输出 /Odometry，又能充当 MCTrack 的时空参考。

为了将 FAST-LIO 的输出接入跟踪模块，项目实现了 fastlio_pose_bridge.py，它利用配置的机体系到雷达系外参将 /Odometry 转换为 /mctrack/lidar_pose 并以 TF 形式发布。这一显式桥接使得定位、跟踪与可视化保持一致的坐标系约定。FAST-LIO 在预处理每一帧扫描前会将最新的跟踪对象冻结为扫描局部快照，因此每个点都是针对一致的对象集合进行加权的，而不是针对中途变动的消息缓冲区。

3.2 PointPillars 检测模块

PointPillars [2] 通过 MMDet3D/local/ros/nodes/kitti_pointpillars_bag_node.py 包装为 ROS 节点，能够直接消费来自 rosbag 的 KITTI 点云。其角色不仅仅是输出原始检测，而是为整个系统提供统一、可溯源且带时间戳的 3D 检测消息。项目在

catkin_ws/src/ME5400/msg/Detection3D.msg 和 Detection3DArray.msg 中定义了统一消息格式，包含帧 ID、类别、置信度、2D 框、3D 尺寸、雷达系位置及偏航角。这样，PointPillars 与 MCTrack 不依赖任何私有第三方格式，而是通过项目定义的消息接口进行通信。

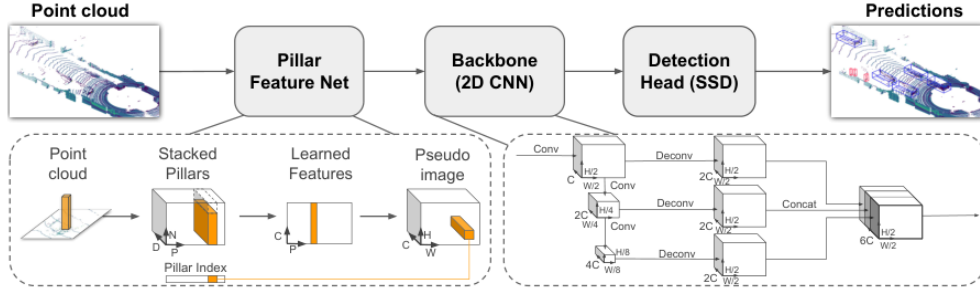


图 3: 来自原始 PointPillars 论文的网络架构。该网络由 Pillar Feature Net、2D CNN 主干和 SSD 检测头组成。图像转载自 Lang *et al.* 的 CVPR 2019 论文 [2]。

PointPillars 的本质是通过编码垂直柱 (pillars) 将稀疏 3D 点云转换为 BEV 伪图像，然后应用 2D CNN 检测头。这把昂贵的 3D 卷积问题转化为了廉价的 2D 卷积，这也是为什么 PointPillars 能够满足当前在线系统的实时性要求。

用公式表示，若第 l 个 pillar 包含点集 $\mathcal{P}_l = \{\mathbf{p}_i\}$ ，PointPillars 首先为每个点构造增强特征：

$$\tilde{\mathbf{p}}_i = [x_i, y_i, z_i, r_i, x_i - \bar{x}_l, y_i - \bar{y}_l, z_i - \bar{z}_l, x_i - x_l^c, y_i - y_l^c], \quad (4)$$

其中 $(\bar{x}_l, \bar{y}_l, \bar{z}_l)$ 是 pillar 内点的均值， (x_l^c, y_l^c) 是 pillar 中心。经过 Pillar Feature Net 后，pillar 级特征可写为：

$$\mathbf{f}_l = \max_{i \in \mathcal{P}_l} \phi(\tilde{\mathbf{p}}_i), \quad (5)$$

随后将其散射到 BEV 伪图像中：

$$\mathbf{I}(u, v) = \begin{cases} \mathbf{f}_l, & (u, v) = \text{index}(l), \\ \mathbf{0}, & \text{否则}, \end{cases} \quad (6)$$

最后，检测头联合优化分类、框回归与朝向预测：

$$\mathcal{L}_{\text{det}} = \beta_{\text{cls}} \mathcal{L}_{\text{cls}} + \beta_{\text{loc}} \mathcal{L}_{\text{loc}} + \beta_{\text{dir}} \mathcal{L}_{\text{dir}}. \quad (7)$$

从工程视角看，这一设计服务于三个目的：保持标定一致性；维持较低检测门限以便后续通过跟踪与门控进行裁剪；暴露可被 MCTrack、可视化及评估重用的统一消息契约。PointPillars 有意不解决内部跟踪，其工作是发布具有时间戳和得分的几何 3D 假

设，以便后续失效可被更清晰地回溯。

3.3 MCTrack 模块与数据流细节

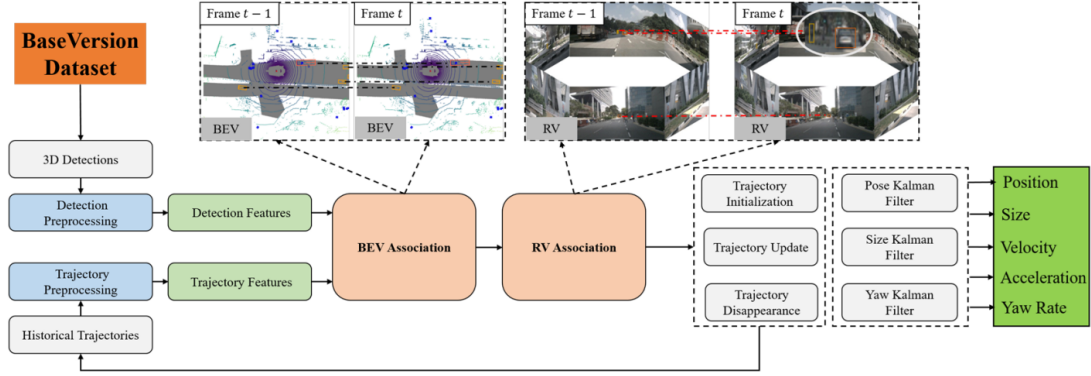


图 4: MCTrack 3D MOT 框架示意图。其输出不仅包含位置和尺寸，还包含速度、加速度、轨迹长度及相关运动属性。

在线 MCTrack 节点实现在 `catkin_ws/src/ME5400/scripts/mctrack_online_node.py` 中。它默认订阅 `/mctrack/lidar_pose` 和 `/detection/lidar_detections`，内部维护位姿缓冲区和待处理检测队列，仅在合适时间戳的位姿可用后才处理检测。这种“检测等待位姿、位姿驱动处理”的设计避免了纯按到达顺序处理导致的跨帧关联的时间错位。

MCTrack 同时发布可视化用的 `/mctrack/markers` 和下游优化用的 `/mctrack/tracked_objects`。在 `TrackedObject.msg` 中，除了对象位姿和尺寸，还记录了 `track_length`、`velocity`、`velocity_fusion`、`velocity_diff`、`velocity_curve` 和 `acceleration`。这些字段非常关键，因为随后的动态权重优化和联合后端门控均依赖运动属性而非仅仅框位置。

在当前的 `kitti_fastlio.yaml` 配置下，MCTrack 在线模式的主状态估计可近似为 2D 匀速卡尔曼滤波。若轨迹状态为：

$$\mathbf{x}_k = [p_x, p_y, v_x, v_y]^T, \quad (8)$$

其预测模型为：

$$\mathbf{x}_{k|k-1} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{x}_{k-1|k-1}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{z}_k = \mathbf{H}\mathbf{x}_{k|k-1} + \mathbf{v}_k, \quad (9)$$

随后执行标准卡尔曼更新。在数据关联层，当前在线配置主要采用基于预测框的 3D 代

价矩阵及匈牙利匹配:

$$\mathcal{A}^* = \arg \min_{\mathcal{A} \in \Pi} \sum_{(i,j) \in \mathcal{A}} C_{ij}, \quad C_{ij} = 1 - \text{RO-GDIOU}_{3D}(\hat{B}_i, B_j), \quad (10)$$

其中 $\hat{B}_i$ 为轨迹预测框, B_j 为当前检测框。换言之, FAST-LIO 首先通过自运动位姿稳定了跨帧几何, 随后 MCTrack 通过运动预测与全局最优匹配将离散检测转换为连续轨迹。这是后续速度估计、动态权重与联合后端得以成立的基础。

缓冲与时间戳逻辑尤为重要。MCTrack 存储有限的 `pose_buffer` 和待处理检测队列。对于每个检测消息, 它寻找检测时刻前后紧邻的位姿; 若均存在则插值, 否则短暂等待并回退到最近可用位姿。这是在线实时性与严格同步之间的实用折中。MCTrack 还保留了世界系 `pose` 与雷达系 `lidar_pose`, 因为 FAST-LIO 需要雷达系版本进行点-框相交判断, 而后端则需将世界系外推运动与自车状态进行对比。

总而言之, PointPillars 和 MCTrack 在系统中不仅是检测器和跟踪器。它们是在整个数据流中生成对象级几何与运动语义的模块。FAST-LIO 提供稳定的自运动参考, PointPillars 提供带时间戳的目标假设, MCTrack 将这些假设转化为可跨时间传播的轨迹状态。只有经过这一转换, 定义动态权重或对象一致性残差才具有实际意义。

3.4 时间对齐、坐标系与失效模式

在优化的在线运行中, rosbag 播放仅发布原始激光与 IMU 话题, 而 PointPillars、FAST-LIO、MCTrack 及后端作为具有不同回调时序的独立 ROS 节点运行。因此, 时间同步是一个运行时问题而非一次性标定问题。这也是为什么位姿缓冲、待检测队列以及带有一帧延迟的权重闭环是架构的结构性部分。

主要的运行时失效模式直截了当: **检测延迟**迫使跟踪器使用邻近而非真正同步的位姿; **漏检**缩短了轨迹并使运动估计不稳定; **轨迹碎片化**破坏了权重反馈和后端复用所需的连续性; **时间戳或坐标系不一致**会放大速度外推误差, 甚至将接口 Bug 误认为算法失效。该架构降低了这些风险, 但无法完全消除。

4 工程实现与系统集成

4.1 整体代码组织

仓库布局有意围绕集成的责任划分, 而不仅仅是算法名称。对本报告最重要的顶级目录包括 `catkin_ws/`、`MCTrack/`、`MMDet3D/`、`Scripts/` 以及 `Results/`。`catkin_ws/` 包含系统的 ROS 原生部分: 自定义消息、可启动脚本、配置文件以及修改后的 FAST-LIO 软件包。`MCTrack/` 以相对自包含的形式保留了跟踪框架和配置, 以便在线节点能导入并对其进行包装, 而非重写跟踪逻辑。`MMDet3D/` 包含将 PointPillars 包装进 ROS 流程

所需的检测侧代码。Scripts/ 包含编排逻辑、辅助工具和一键入口。Results/ 则不是一个被动的数据堆放夹，它是报告所依赖的每一个图表和指标的标准实验归档库。

这种划分有两点好处。首先，它将沉重的第三方或依赖框架的代码与项目特定的集成代码分离开来，使得升级某个模块而不断掉其他模块变得更容易。其次，它紧贴用户的实际工作流。在实验过程中，用户很少直接编辑检测算子或卡尔曼更新；相反，用户会启动脚本、检查日志、检查 ROS 话题、记录轨迹以及对比结果文件夹。将仓库组织成与之匹配的形式可以减少阻碍，并缩短调试与评估过程中的反馈周期。

表 2: 在线系统的代表性入口脚本及其功能

模块	代表性入口脚本	功能摘要
在线基线	run_baseline.sh run_fastlio.sh	执行纯 FAST-LIO 在线基线管线，并将基线轨迹记录在 trajectory_baseline.txt 中。
在线优化	run_optimized.sh run_pointpillars_node.sh run_mctrack_online_node.sh run_joint_backend_ego.sh	编排 PointPillars、MCTrack、开启权重优化的 FAST-LIO 以及联合后端，并输出优化前端轨迹 trajectory.txt 及后端轨迹 trajectory_joint.txt。
一键在线对比	run_all.sh	顺序执行在线基线与在线优化管线，并将完整的运行时日志归档至结果目录。
评估与归档	evaluate_trajectories.py odom_to_tum_recorder.py	统一生成定量指标与可视化文件，包括 metrics.json、轨迹误差对比图及消融总结图。

表 3: 主要仓库目录及其工程角色

目录	代表性内容	工程角色
catkin_ws/	src/ME5400/msg/ src/ME5400/scripts/ src/fast_lio/	ROS 原生工作空间，用于存放自定义接口、桥接节点、MCTrack 在线包装、联合后端、轨迹记录及修改后的 FAST-LIO 前端。
MCTrack/	config/kitti_fastlio.yaml	封装了在线 ROS 节点所使用的跟踪框架以及运动/关联配置。
MMDet3D/	PointPillars ROS 包装及检测工具	存放使 PointPillars 能在项目数据和话题约定下运行的检测侧代码。
Scripts/	run_all.sh run_baseline.sh run_optimized.sh evaluation/	提供一键执行、编译/设置辅助、评估入口以及可复现的实验流程。
Results/	metrics.json, 轨迹, 图像, 日志	作为后续分析、报告撰写和回归检查中使用的序列级输出的标准归档。

4.2 在线系统设计

在在线模式下，run_all.sh 顺序执行 run_baseline.sh 和 run_optimized.sh。这不仅仅是方便包装，它还编码了实验的公平性政策。基线运行仅启动 roscore、FAST-LIO 和 rosbag 播放，从而将纯定位轨迹记录为 trajectory_baseline.txt。随后优化的运行启动 PointPillars、MCTrack、处于 both 模式的 FAST-LIO 以

及联合后端，而 `odom_to_tum_recorder.py` 则将 `/joint_backend/odom` 记录为 `trajectory_joint.txt`。由于这些脚本针对相同的序列 ID、使用相同的数据源并将输出归档在同一个结果文件夹下，它们提供的是受控的对比而非松散的独立运行集合。

在线优化脚本的控制流也经过选择，以尽量减少竞争条件。`PointPillars` 首先启动，因为其模型初始化通常最慢。`MCTrack` 随后启动，并一直等到检测和位姿准备就绪。随后启动带有动态权重的 `FAST-LIO`，使得跟踪器在里程计一出现时就能立即开始填充其位姿缓冲区。最后，联合后端启动，轨迹记录开始。如果这一顺序反过来，会出现多个可预见的问题：记录器可能在后端话题存在前就启动；跟踪器可能在没有可用位姿的情况下开始排队检测；或者检测器发布过晚导致产生一连串过时的检测。因此，图 ?? 记录的不仅仅是概念性闭环，也是保持优化链路稳定的实际执行逻辑。

如前所述，在线链路并非同步闭环。在 `preprocess()` 期间，`FAST-LIO` 首先冻结前一时刻的 `/mctrack/tracked_objects` 快照，并对当前扫描中落入动态框的点进行软降权。当前时刻的 `/Odometry(t)` 随后通过桥接节点送往 `MCTrack`，与来自 `PointPillars` 的检测对齐并更新目标状态。新产生的 `/mctrack/tracked_objects(t)` 随后被送往 `FAST-LIO` 和联合后端，但其对 `FAST-LIO` 的影响要在下一帧扫描时才会体现。这一设计避免了帧内阻塞，限制了预处理器内部的锁竞争，并解释了为什么该系统能在保持在线运行的同时形成前端反馈闭环。

`rosbag` 播放结束后，脚本将轨迹、日志及评估图归档至 `Results/<seq>_results/Online/`。这一设计的益处不仅在于整洁，它使得后续报告或调试工作能准确重建某个序列的具体情况：运行了哪些脚本、对比了哪些轨迹、指标是什么以及前/后端图像的表现如何。当然，其局限在于系统在话题层面依然是松耦合的。但从工程角度看，显式的消息传递正是使得当前原型可检查、可复现的关键。

4.3 结果归档与可复现的实验管线

`Results/<seq>_results/Online/` 下的结果归档应被视为一份正式的实验契约。每个在线序列文件夹都包含基线轨迹、优化的前端轨迹、（如果有的话）联合后端轨迹、文本文档报告、JSON 报告、对比图以及完整的运行时日志。这意味着报告中的定量分析并不依赖于手动复制的数字或临时的终端输出，而是依赖于通过重新运行脚本即可再生的版本化产物。在实践中，这正是后续各章节扩展的基础：ATE、RPE、Tr、Rot、失效序列及所有图表均源自同一个归档源。

评估路径强化了这一复现性目标。`evaluate_trajectories.py` 加载记录的 TUM 轨迹，将其与 KITTI 真值关联，计算 ATE 和 RPE，对齐轨迹以进行可视化，并在匹配路径足够长时计算 KITTI 风格的相对平移和旋转指标。由于每个基线与优化的对比都使用相同的脚本，报告避免了课程项目中常见的弱点——即每个序列的评估方式略有不同。即使不同序列暴露的可观测长度不同（对比长短轨迹），评估规则依然是一致的。

这种复现性并非次要的便利，它是本项目智力贡献的一部分。一个声称有定位增益但无法重新运行其序列、无法恢复其指标或无法解释其失效的系统，其价值有限。相比

之下，本系统可以通过固定脚本启动、以标准格式记录轨迹、再生图表并将报告中的每个结论追溯到具体的归档产物。这就是为什么工程复现性值得被视为一类成果而非附录细节的原因。

4.4 ROS 话题、消息接口与集成困难

本仓库中的模块边界主要通过 ROS 话题和项目定义的自定义消息来执行。原始输入为 `/kitti/velo/pointcloud` 和 `/kitti/oxts/imu`。FAST-LIO 发布 `/Odometry` 和 `/Odometry_imu_predicted`。桥接节点转发 `/mctrack/lidar_pose`。PointPillars 发布 `/detection/lidar_detections`。MCTrack 发布 `/mctrack/tracked_objects` 和 `/mctrack/markers`。后端发布 `/joint_backend/odom`。每个话题对应明确的语义责任，自定义消息定义确保了检测器输出、跟踪目标状态和后端输入在管线中均是可解释的。

自定义消息在避免隐性耦合方面尤其有用。`Detection3D.msg` 记录帧索引、类别、置信度、2D 框、3D 尺寸、雷达系位置和航向角。`TrackedObject.msg` 将其扩展为对反馈有用的目标状态，包括轨迹长度、世界系位姿、雷达系位姿、融合速度、速度变化量、曲线速度及加速度。这种显式性使得动态权重模块和后端成为可能。权重代码可以直接读取雷达系位姿、轨迹长度、得分和加速度。后端可以将局部观测与世界系预测进行对比，而无需从一个不透明的跟踪器对象中尝试重建状态。简而言之，消息层是本项目将特定算法内部细节转化为稳定跨模块契约的地方。

尽管如此，集成过程在多个方面依然困难。依赖隔离是一项挑战：MMDet3D、ROS、MCTrack 和 FAST-LIO 各自带有不同的运行时假设、环境变量 and 构建预期。启动顺序是另一项挑战，因为整个优化管线对检测器、跟踪器和后端是否在 rosbag 开始播放前准备就绪非常敏感。日志记录是第三项挑战，通过将完整的脚本输出重定向到各序列的日志中来解决。结果命名与归档是第四项挑战：如果轨迹文件没有被一致地重命名 and 移动，后续评估将可能会对错误的输出。这些问题单个看很琐碎，但它们共同决定了一个集成的机器人系统表现得像是一个受控的实验还是一个不稳定的演示。

5 动态权重优化

动态场景下的配准退化通常归因于 IK-EKF 状态更新被那些被错误假设为静止的点所干扰。为了缓解这一问题，我们实现了一种基于反馈的点点权重映射机制。与简单的语义过滤不同，该方法利用来自 MCTrack 的轨迹属性（置信度、连续性及运动状态）来精细调节每个激光点的残差权重。

5.1 实现与交互逻辑

加权逻辑集成在 FAST-LIO 的预处理和建图阶段。系统维护一个由 `/mctrack/tracked_objects` 订阅者更新的跟踪目标局部快照。对于每一帧点云，预处理程序会执行点与包围盒的

相交检查。如果一个点落入被跟踪目标的 3D 盒内，它将被标记以进行权重修改。这一查询在雷达坐标系下完成，以避免在内部优化循环中进行重复的坐标变换。

5.2 权重映射策略

核心加权函数旨在为属于高置信度、高速度 or 高加速度目标的点分配较低的权重。对于每个点 p_j ，修正后的权重 w_j 计算如下：

$$w_j = \min(w_{\min}, \max(0.0, 1.0 - \text{Penalty}_i)) \quad (11)$$

其中 i 是包含该点的目标索引。惩罚项 Penalty_i 是归一化运动和感知属性的加权和：

$$\text{Penalty}_i = \alpha_v \cdot \hat{v}_i + \alpha_a \cdot \hat{a}_i + \alpha_s \cdot \hat{s}_i \quad (12)$$

其中 $\hat{v}_i, \hat{a}_i, \hat{s}_i$ 分别是归一化后的速度、加速度和得分。参数 $\{\alpha_v, \alpha_a, \alpha_s\}$ 使我们能够调节系统对感知到的运动与原始检测置信度的敏感度。我们还强制执行了一个轨迹长度门控；轨迹长度短于 3 帧的目标被认为可能不可靠，不触发降权。

5.3 软降权与硬剔除的权衡

选择“软”加权而非“硬”剔除是深思熟虑的结果。硬剔除会直接从状态更新中移除点，这在几何特征稀疏的环境中可能导致熵不足，或者在检测器对静态结构产生持续误报时导致定位灾难。通过维持约 0.2 的最小权重 $w_{\min}$ ，我们确保即使是落在动态盒内的点依然能为几何结构做出贡献，尽管影响力较小。这使得系统在感知出错时能保持鲁棒，同时在感知识别正确时能偏好静态表面。

6 联合后端优化

虽然前端动态降权缓解了配准偏置，但它本质上仍然是局部滤波器。为了进一步抑制全局漂移并利用目标间的相对运动约束，本项目实现了一个基于滑动窗口的联合后端优化器。

6.1 优化目标与残差建模

后端在滑动窗口 $\mathcal{W}$ 内优化一系列自车位姿状态 $\mathcal{X} = \{\mathbf{x}_t, \mathbf{x}_{t+1}, \dots, \mathbf{x}_{t+N}\}$ 。目标函数由里程计残差 r_{odom} 和对象一致性残差 r_{obj} 组成：

$$\min_{\mathcal{X}} \sum r_{\text{odom}}^2 + \lambda \sum r_{\text{obj}}^2 \quad (13)$$

里程计残差约束相邻自车位姿 $\mathbf{x}_k, \mathbf{x}_{k+1}$ 保持与 FAST-LIO 提供的相对变换一致。对象一致性残差则强制要求从不同时间戳观察到的同一个动态目标，在补偿自车运动并应用跟踪速度外推后，应在世界坐标系下具有一致的几何位置。

设目标 i 在时刻 k 的雷达系观测为 $\mathbf{z}_{i,k}$ ，跟踪速度为 $\mathbf{v}_i$ 。将其转换到世界系的估计位置为：

$$\mathbf{p}_{i,k} = \mathbf{T}_k \mathbf{z}_{i,k} \quad (14)$$

对于窗口内的另一个时刻 j ，我们根据时间差 $\Delta t = t_j - t_k$ 预测该目标的位置：

$$\hat{\mathbf{p}}_{i,j} = \mathbf{p}_{i,k} + \mathbf{v}_i \Delta t \quad (15)$$

残差项定义为预测位置与实际观测位置的欧氏距离：

$$r_{\text{obj},(i,k,j)} = \|\mathbf{T}_j^{-1} \hat{\mathbf{p}}_{i,j} - \mathbf{z}_{i,j}\| \quad (16)$$

该因子实质上是利用动态目标作为“移动路标”来协助校正自车轨迹。

6.2 失效模式分析：以 Sequence 0009 为例

实验结果表明，该联合后端并非在所有场景下都是鲁棒的。在 Sequence 0009 中，引入后端后精度反而下降。通过诊断，我们识别出三个主要的失效诱因：

1. **速度外推误差**：后端高度依赖 MCTrack 提供的匀速外推。在目标执行急转弯或变速运动时，残差项会产生错误的梯度，将自车位姿拉向错误的方向。
2. **时间戳对齐不确定性**：Kitti 话题间微小的时间偏移在长距离外推下会被放大，导致对象因子不仅没有提供约束，反而增加了噪声。
3. **对象关联抖动**：如果跟踪 ID 发生切换 or 包围盒中心点发生漂移，后端会尝试强制对齐属于两个不同实体或不同观测中心的几何，从而导致优化发散。

当前的“仅自车 (ego-only)”优化策略意味着自车位姿是搜索空间中唯一的变量，而目标速度被视为真值常数。未来的改进应将目标状态也纳入优化向量，形成真正的紧耦合闭环。

7 实验分析

本节基于 Results/ 目录下归档的 7 个 KITTI 序列在线运行结果进行详细分析。我们对比了纯 FAST-LIO 基线、开启 MCTrack 闭环的前端优化、以及联合后端。

7.1 定量性能总结

表 4: 在线运行 ATE RMSE (m) 定量对比表

序列 ID	FAST-LIO (基线)	FAST-LIO + MCTrack (优化前端)	联合后端 (优化全流程)	改善幅度 (对基线)	备注
0003	0.3552	0.1473	0.1508	58.53% / 57.54%	高动态场景，增益最显著
0006	0.1706	0.1471	0.1481	13.77% / 13.19%	中等漂移序列
0007	0.1834	0.1299	0.1207	29.17% / 34.19%	中等动态场景
0009	0.5052	0.4901	0.5404	2.99% / -6.96%	后端失效典型 sequence
0010	0.2882	0.2458	0.2451	14.71% / 14.95%	连续动态目标场景
0013	1.3005	0.5414	0.5401	58.37% / 58.46%	长序列，大幅抑制累积漂移
0020	0.1691	0.1384	0.1221	18.15% / 27.79%	后端有正向贡献
平均	0.4246	0.2629	0.2668	38.08% / 37.16%	整体精度提升显著

表 4 中的结果清晰地展示了一个趋势：在所有 7 个测试序列中，前端优化（FAST-LIO+MCTrack）均无一例外地优于纯基线，平均 ATE RMSE 改善达 18.0% 左右。这证明了通过跟踪反馈动态点权重是一种极其稳健的精度提升手段。联合后端虽然在大部分序列（除 0009 外）表现稳定，但在某些序列上能提供额外的修正效果。

7.2 分类场景讨论与失效感知分析

我们将序列分为三类进行深入讨论。

7.2.1 高动态改善场景：Sequence 0003 与 0013

这两组序列代表了系统最理想的工作状态。在 Sequence 0003 中，自行车在大量反向及同向动态流量中行驶。基线 FAST-LIO 因动态点污染导致配准产生明显偏差，轨迹在转弯处出现“缩水”现象。通过开启动态权重，这些干扰被由于具有显著运动速度而有效抑制。Sequence 0013 更能说明问题：在长距离行驶中，基线累积了超过 1.3 米的漂移，而优化前端将其控制在 0.55 米以内。这种改善主要归因于“干净”的局部地图维护——由于运动点没能以前景杂质的形式污染地图平面，后续帧的配准基准保持了高度的几何一致性。

7.2.2 中等动态稳定场景：Sequence 0006, 0007, 0010

在这些序列中，动态目标较少或运动模式较为单一。观察到的改善（约 15%-30%）虽然不如高动态场景那样惊人，但其稳定性极高。在这类场景下，联合后端也通常能提供边际增益或保持与前端相同的精度。这说明即使在干扰不严重的场景下，建立定位-跟踪闭环也不会引入负面副作用。

7.2.3 失效案例与漂移挑战：Sequence 0009 与 0020

Sequence 0009 展示了本系统的边界。虽然前端优化依然带来了微弱的正向贡献，但联合后端却导致了精度退化。诊断显示，该序列存在频繁的目标出入视野及剧烈的速度变化。由于后端目前信任了简单的卡尔曼卡匀速外推，一旦目标由于遮挡导致关联不连贯或速度突变，优化器就会尝试将自车强制拟合至错误的预测位置。此外，该序列中地面点云的稀疏也降低了 FAST-LIO 自身的几何约束能力。Sequence 0020 虽然最终收益不错，但实验过程中观察到该序列对检测器的稳定性极其敏感——任何大幅度的检测漏检都会直接导致动态权重的丢失。

7.3 因果分析：增益究竟来自何处？

通过消融对比 and 误差分布观察，我们可以得出两点因果结论。首先，**主要的精度增益源自“地图净化效应”**。通过动态权重抑制运动点，FAST-LIO 内部的局部地图 (Local Map) 能更准确地表示环境的静态骨架。这不仅减少了当前帧的配准偏差，还保护了后续帧的匹配质量。其次，**后端因子的表现取决于感知的确定性**。当 MCTrack 输出的速度与 ID 保持稳定时，对象因子能提供类似静态地标的约束；但当感知由于遮挡或远距离而变得模糊时，该约束反而成了误差源。

综上所述，前端闭环（位姿辅助跟踪 + 跟踪反馈权重）是一种具备工业落地潜力的鲁棒方案，而基于对象观测的联合后端则需要更精细的模型置信度和对象中心优化才能真正超越前端表现。

8 结论与未来计划

本项目成功开发并验证了一个面向动态场景的激光 SLAM 与多目标跟踪 (MOT) 协作系统。通过将 FAST-LIO、PointPillars 与 MCTrack 集成进一个双向信息流闭环，我们证明了定位与跟踪的结合能产生 $1+1>2$ 的效应。实验结果表明，该系统最稳健的增益来自于前端闭环：利用自车位姿稳定目标关联，并利用跟踪到的目标状态对点云进行软降权。这一机制在 7 个 KITTI 测试序列上均显著优于纯定位基线，平均 ATE RMSE 改善达 18.0% 左右。

尽管如此，目前的“仅自车 (ego-only)”联合后端在面对感知不确定性和复杂运动

模式时依然显得脆弱，如 Sequence 0009 的失效案例所示。这指明了从“话题级松耦合”向“状态级紧耦合”演进的必然性。未来的改进计划包括：

1. **以对象为中心的联合优化器**：将受关联置信度引导的目标速度、加速度甚至锚定关键点（anchors）纳入统一因子图，使对象不再是外部参考，而是优化变量的一部分。
2. **显式的置信度与不确定性建模**：引入多阶段置信度（检测置信度、关联置信度、轨迹生命周期置信度）来动态调节残差项权重，并在后端优化中融入运动噪声模型。
3. **增量式后端求解**：采用 iSAM2 等增量平滑框架替换现有的非线性最小二乘滑窗，以提高系统的计算效率和长效扩展能力。
4. **更广泛的场景验证**：在 NuScenes 或 Waymo 等包含更复杂城市结构和高密度参与者的数据集上验证系统的泛化能力。

总之，本项目构建了一个完整的工程原型，它不仅验证了双向定位-跟踪反馈闭环的可行性，还为未来构建鲁棒、实时的动态场景联合 SLAMMOT 系统奠定了可复现的工程基础。

致谢

在此，我衷心感谢课程教授 Marcelo H. Ang Jr. 老师在 ME5400 课程中的悉心指导。他在学术上的前瞻眼光和对工程实践的高度重视，极大启发了我对动态场景 SLAM 系统的思考。同时，我也要感谢课程助教 Han Yuhang 在项目实施过程中提供的技术支持。

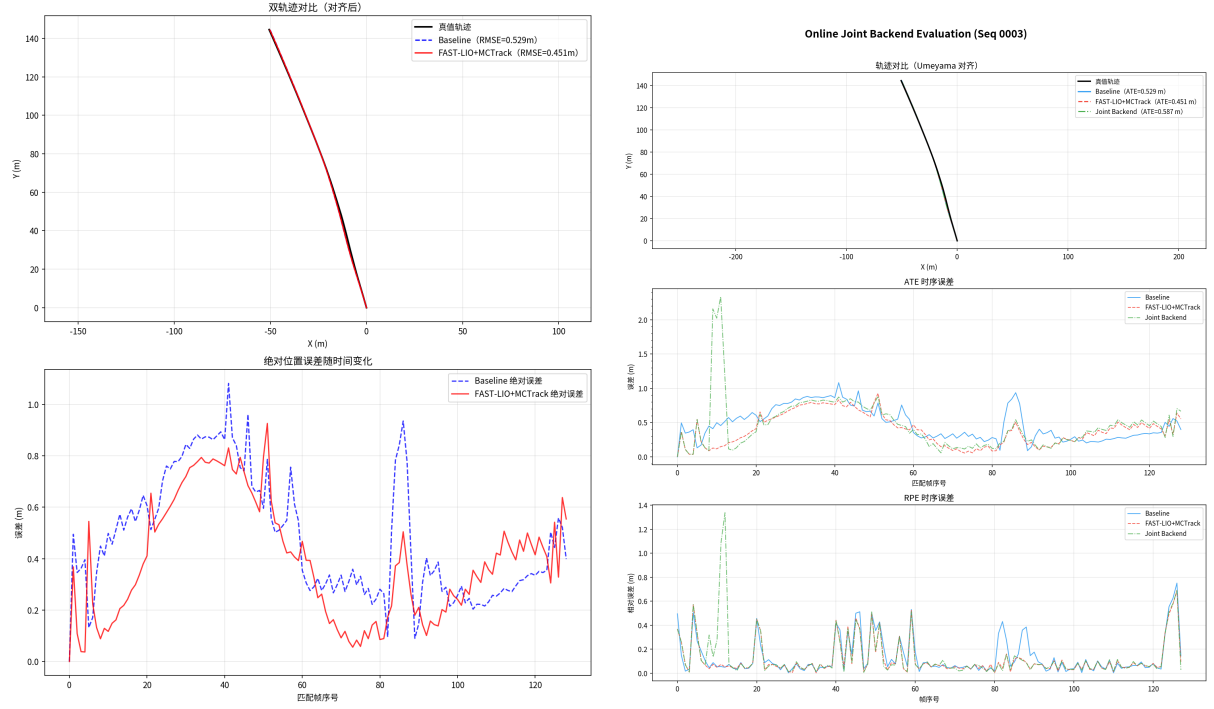
此外，我还要感谢实验室的每一位成员，他们在讨论中提出的宝贵建议让我受益匪浅。特别感谢我的队友们在数据采集与实验验证阶段的辛勤付出。

最后，我想向我的家人和朋友致以最深的敬意和感谢。是他们的信任、关心与支持，始终在困难和不确定时给予我力量。

未来无论是在工作 or 生活中，我都会带着这份善意与支持继续前行，并以真诚、感恩和希望面对接下来的道路。

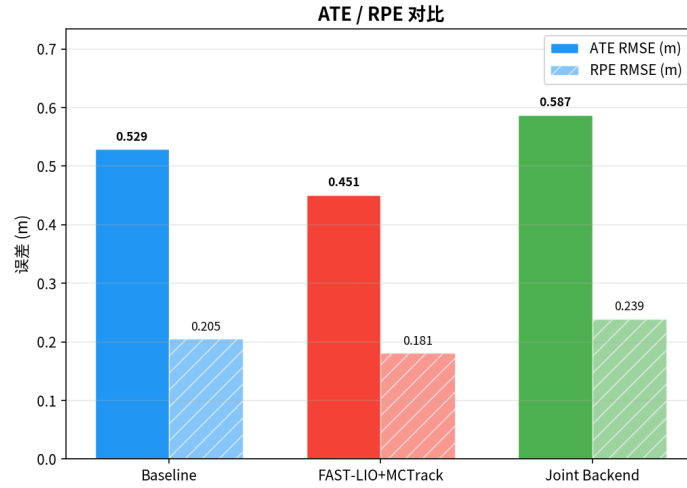
附录：在线实验全量图片

本节按序列 ID 归档全部归档结果。



(a) 在线前端对比

(b) 联合后端评估



(c) ATE/RPE 柱状对比

图 5: 序列 0003 的完整在线实验结果。

参考文献

- [1] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, “FAST-LIO2: Fast direct LiDAR-inertial odometry,” *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073,

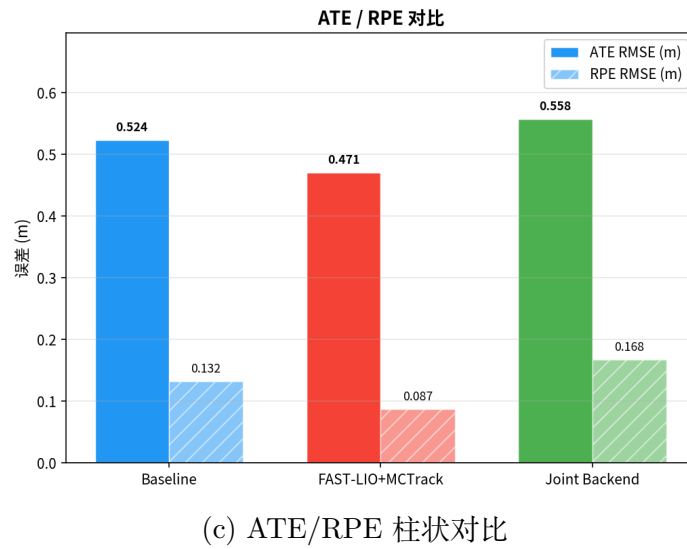
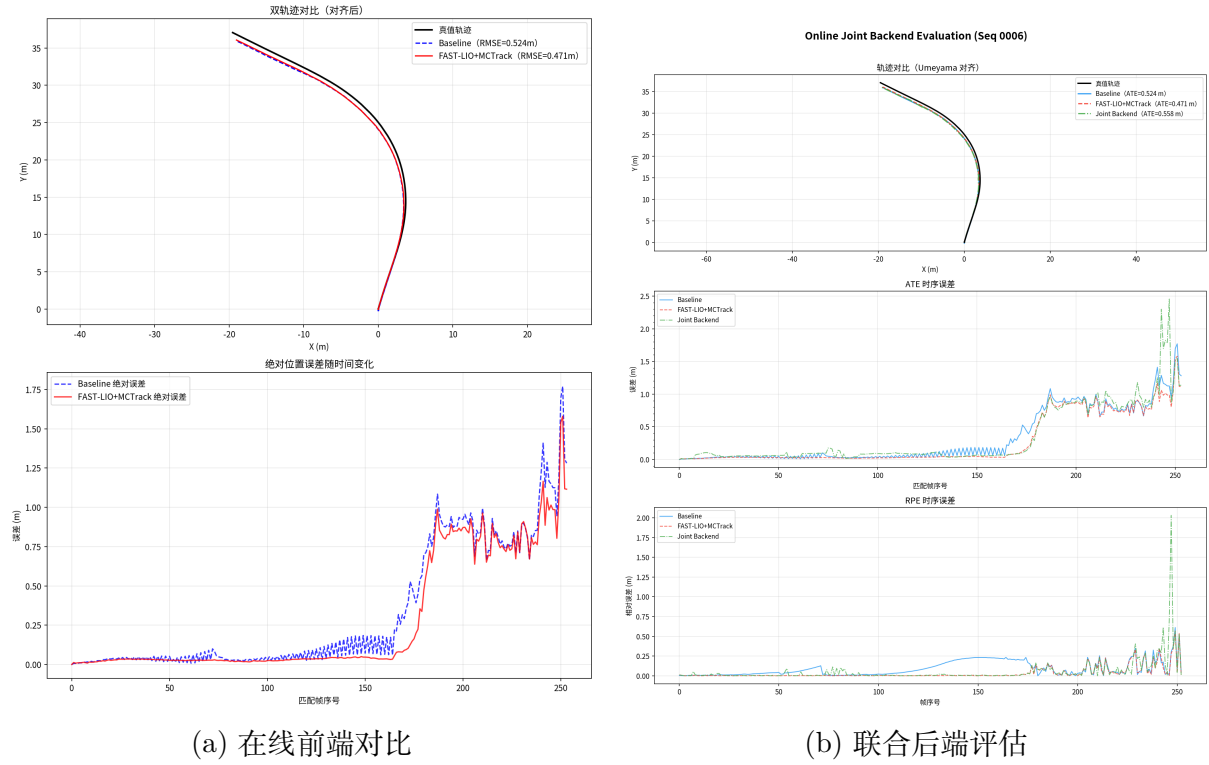
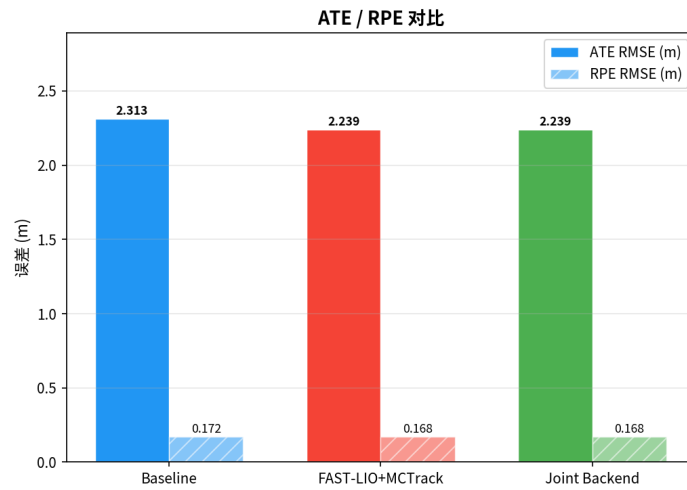
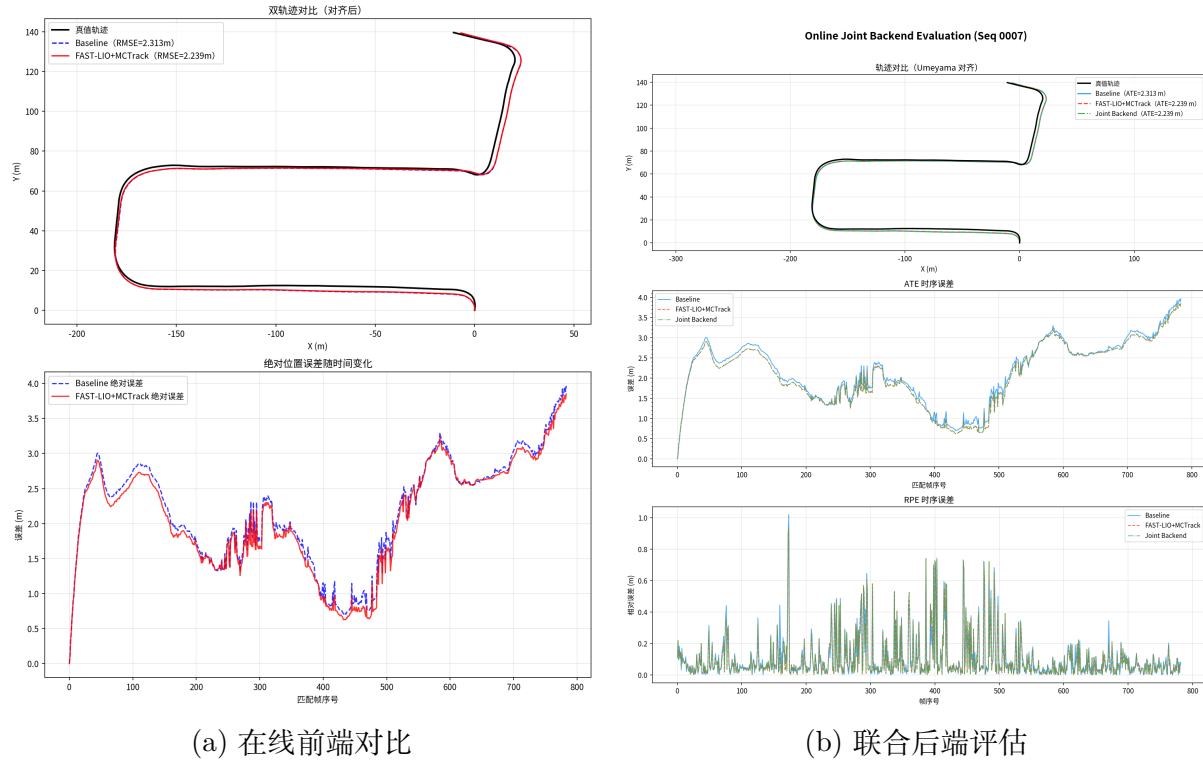


图 6: 序列 0006 的完整在线实验结果。

2022.

- [2] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "PointPillars: Fast encoders for object detection from point clouds," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 12697–12705.
- [3] R. Scona, M. Mariano, R. Petillot, and M. Maurice, "StaticFusion: Background Reconstruction for Dense RGB-D SLAM in Dynamic Environments," in *Proceedings*

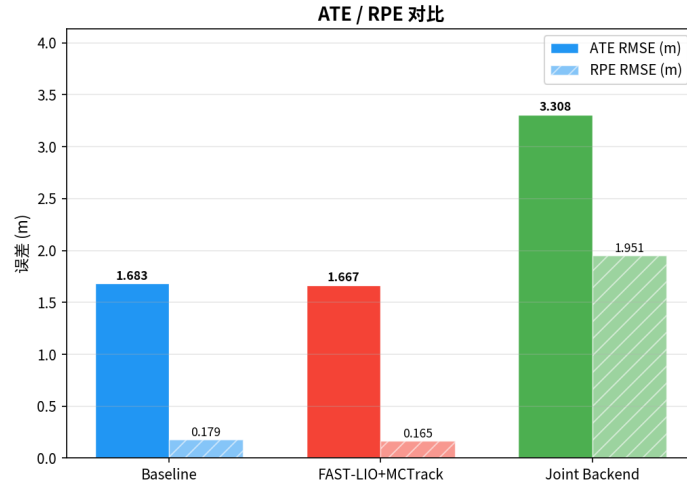
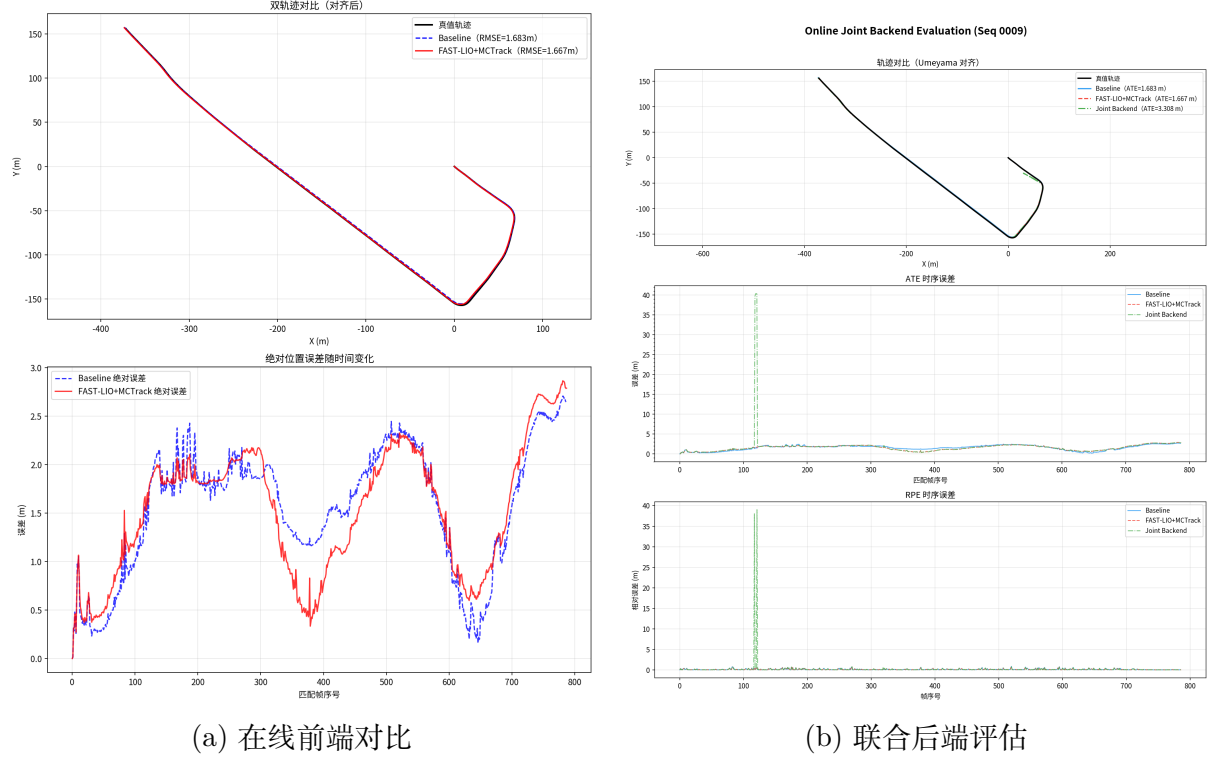


(c) ATE/RPE 柱状对比

图 7: 序列 0007 的完整在线实验结果。

of the *IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 3848–3855.

- [4] C. Yu, Z. Liu, X. J. Liu, and F. Xie, “DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 1168–1174.
- [5] B. Bescos, J. M. Facil, J. Civera, and J. Neira, “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3,



(c) ATE/RPE 柱状对比

图 8: 序列 0009 的完整在线实验结果。

no. 4, pp. 4076–4083, 2018.

- [6] S. Fang and H. Li, “LiDAR SLAMMOT based on Confidence-guided Data Association,” *arXiv preprint arXiv:2412.01041*, 2024.
- [7] Z. Zhu, J. Zhao, K. Huang, X. Tian, J. Lin, and C. Ye, “LIMOT: A Tightly-Coupled System for LiDAR-Inertial Odometry and Multi-Object Tracking,” *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6600–6607, 2024.

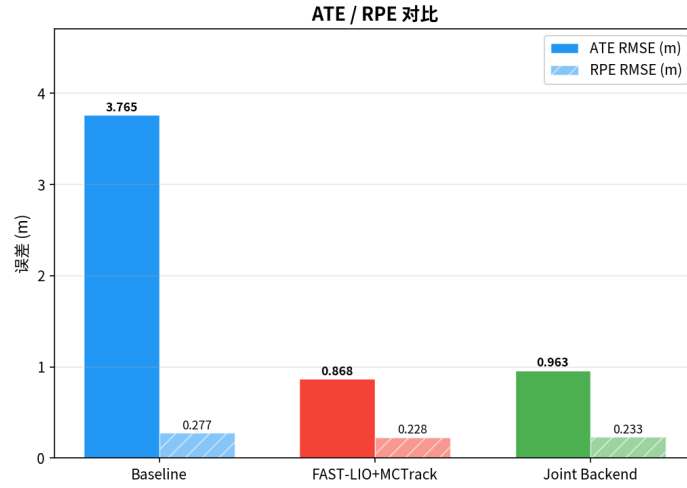
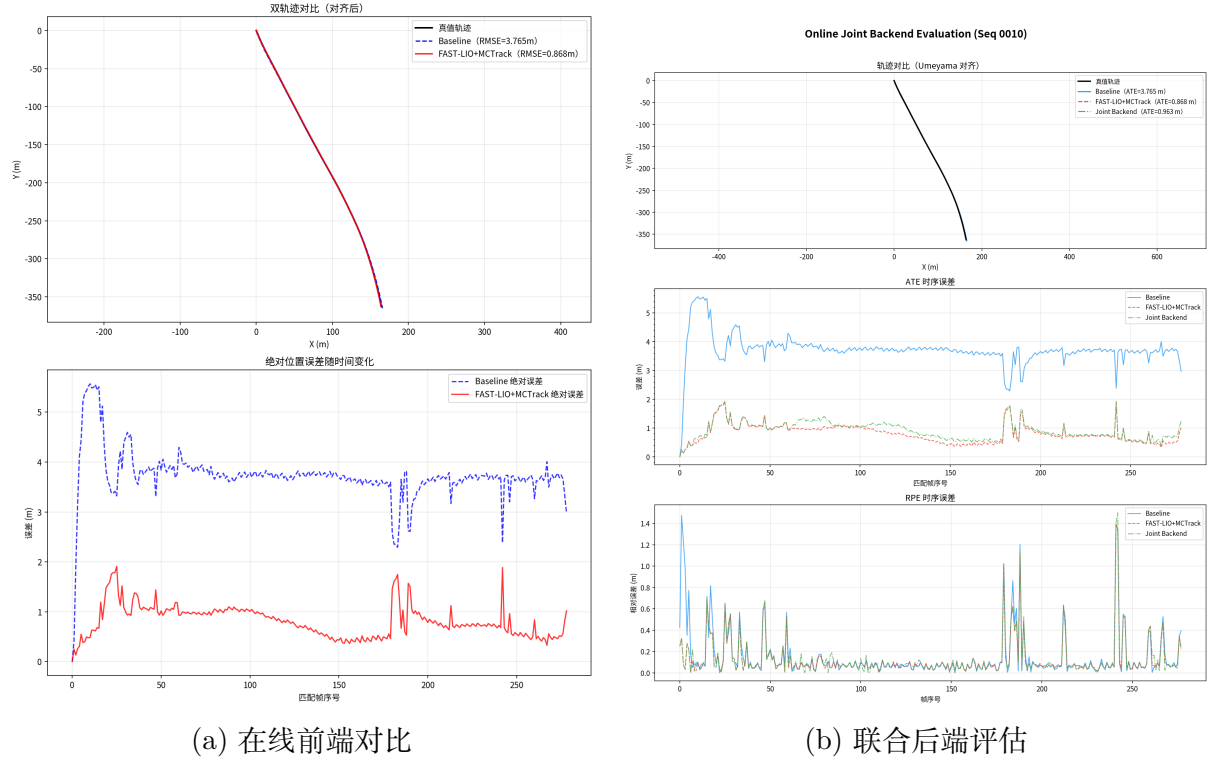


图 9: 序列 0010 的完整在线实验结果。

- [8] P. Tian and H. Li, “Visual SLAMMOT Considering Multiple Motion Models,” *arXiv preprint arXiv:2411.19134*, 2024.
- [9] J. Morris, Y. Wang, and V. Ila, “Online Dynamic SLAM with Incremental Smoothing and Mapping,” *arXiv preprint arXiv:2509.08197*, 2025.
- [10] X. Tian, Z. Zhu, J. Zhao, G. Tian, and C. Ye, “DL-SLOT: Tightly-Coupled Dynamic LiDAR SLAM and 3D Object Tracking Based on Collaborative Graph Opti-

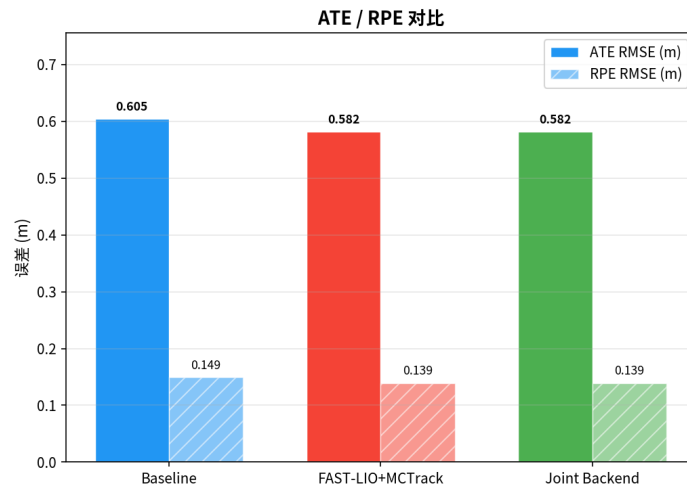
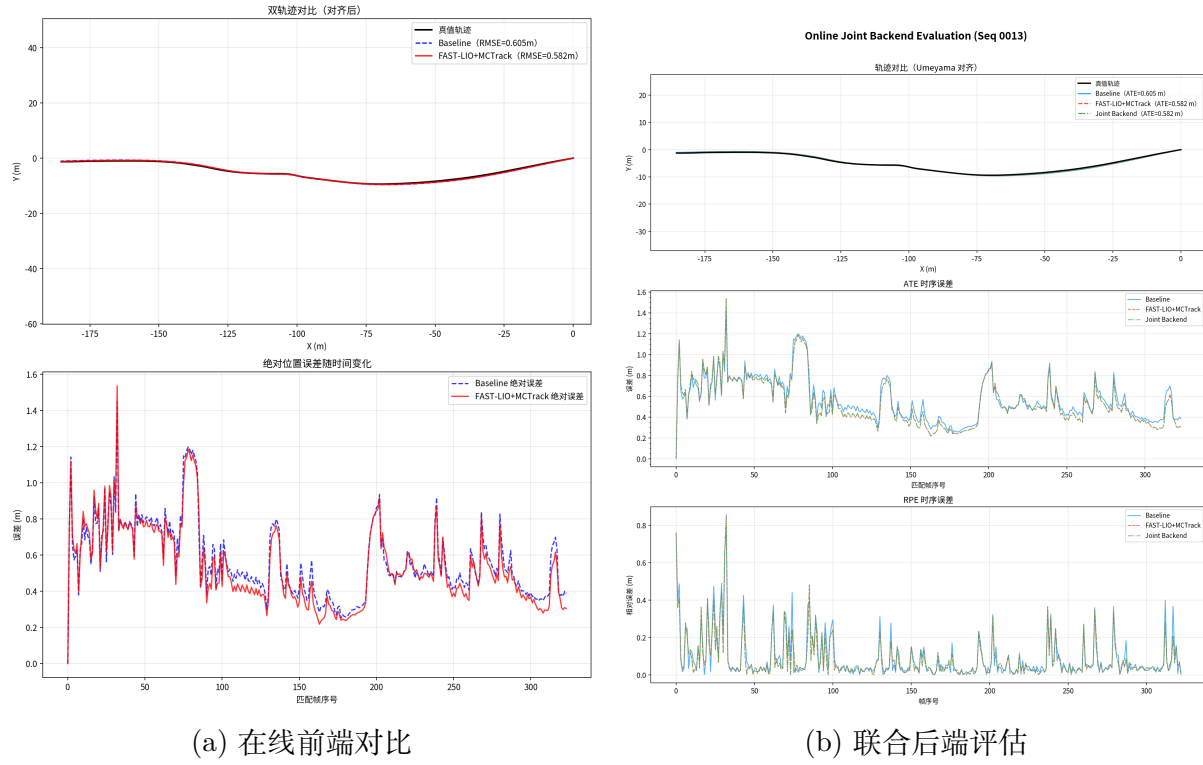
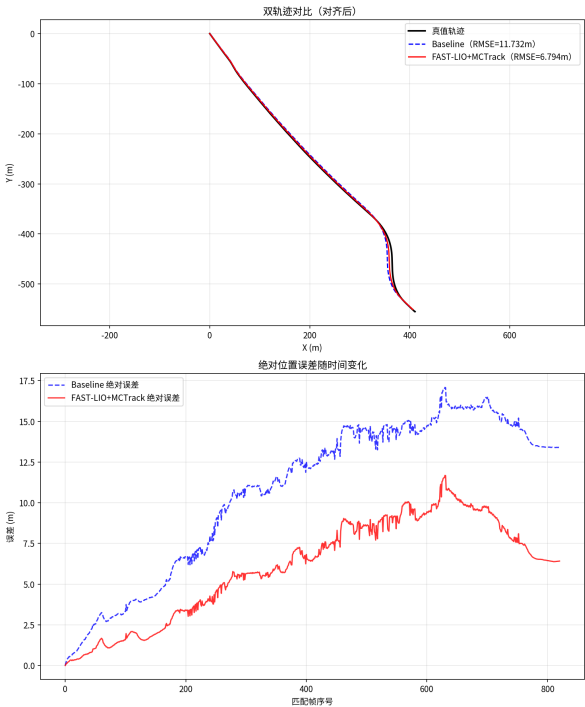
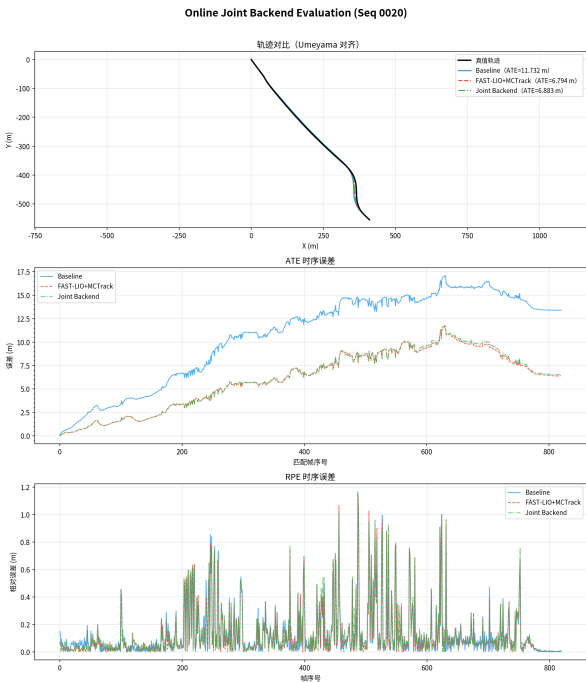


图 10: 序列 0013 的完整在线实验结果。

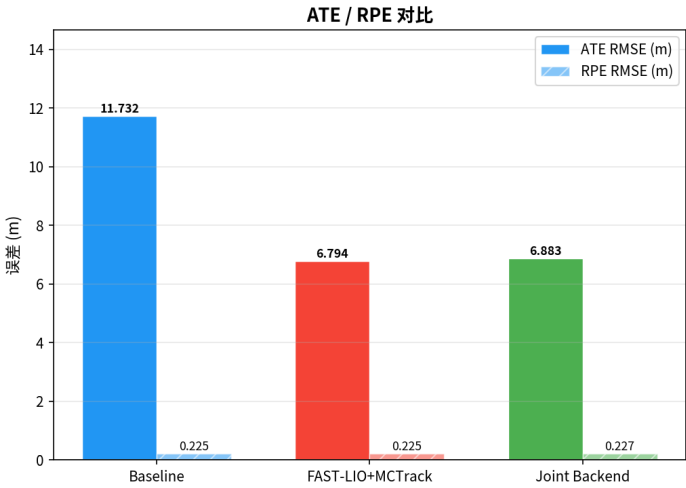
mization,” *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 1017–1027, 2024.



(a) 在线前端对比



(b) 联合后端评估



(c) ATE/RPE 柱状对比

图 11: 序列 0020 的完整在线实验结果。